

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



SUBJECT: SOFTWARE ARCHITECTURE
PROJECT FINAL REPORT: MOVIE TICKET BOOKING SYSTEM

Instructor :	Vu Quang Dung
Student Name:	Vu Duc Hieu
Student ID:	23010226
Email:	23010226@st.phenikaa-uni.edu.vn
Course class:	CSE703110-1-2-25(N01)

Ha Noi, March 2026

1. UI/UX DESIGN & FRONTEND ARCHITECTURE.....	4
1.1 Architectural Vision and Technology Stack	4
1.2 Single Page Application (SPA) and Client-Side Routing	4
1.3 Component-Based Structural Design	5
1.4 State Handling and Reactive UI Updates	5
1.5 Responsive Design and Accessibility.....	6
2. CLIENT INTEGRATION - THE API CLIENT LAYER.....	7
2.1 Centralized Communication Strategy.....	7
2.2 Secure Header Management and Gateway Authentication.....	8
2.3 Standardized CRUD Method Wrappers	8
2.4 Robust Error Handling and Session Recovery	8
2.5 Data Transformation and JSON Parsing	9
3. ADMIN PAGES - MANAGEMENT MODULES & CRUD OPERATIONS ...	10
3.1 The Admin Dashboard Ecosystem	10
3.2 Movie Management and Resource CRUD	11
3.3 Showtime and Room Management	12
3.4 Dynamic Table Rendering and Search Optimization.....	13
3.5 Form Validation and Data Integrity	14
3.6 Analytics and System Overview	14
4. USER PAGES - CUSTOMER JOURNEY AND INTERACTIVE BOOKING FLOW	15
4.1 The "Cinema First" Visual Experience	15
4.2 Movie Details and Information Hierarchy	16
4.3 The Interactive Booking Flow (The Core Engine).....	17
4.4 Payment Integration and Security	18
4.5 User Profile and "My Bookings"	18
5. SUMMARY OF CONTRIBUTIONS	19

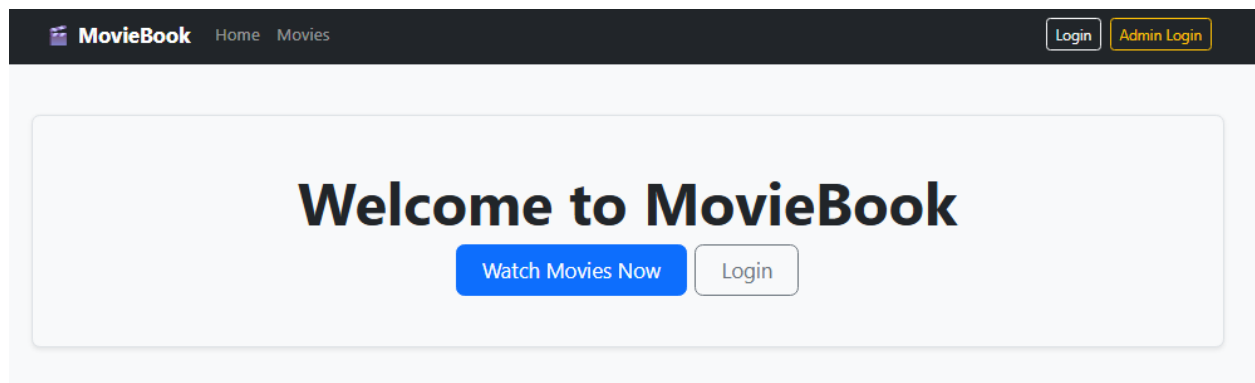
5.1	Technical Excellence and Architectural Integrity	19
5.2	Robust Security and Seamless API Integration.....	19
5.3	Administrative Empowerment and Data Management	19
5.4	User-Centric Design and Conversion Optimization.....	20
5.5	Conclusion of Impact	21

1. UI/UX DESIGN & FRONTEND ARCHITECTURE

1.1 Architectural Vision and Technology Stack

The primary objective for the frontend development was the creation of a modern, high-performance, and "app-like" experience using a Single Page Application (SPA) architecture. To achieve this without the overhead of heavy frameworks, a lightweight and optimized stack was selected:

- **Vite:** Utilized as the next-generation build tool to provide rapid Hot Module Replacement (HMR) during development and highly optimized bundling for production.
- **Vanilla JavaScript (ES6+):** Employed to maintain direct control over the Document Object Model (DOM), ensuring maximum execution speed and zero library-induced latency.
- **Bootstrap 5:** Implemented for its robust grid system and pre-built UI components, ensuring a professional appearance and cross-browser compatibility.
- **Custom CSS:** Written to provide the unique "Cinema" aesthetic, including dark-mode optimizations, glassmorphism effects for cards, and interactive hover states for seat selection.



1.1: Overview showing the Homepage

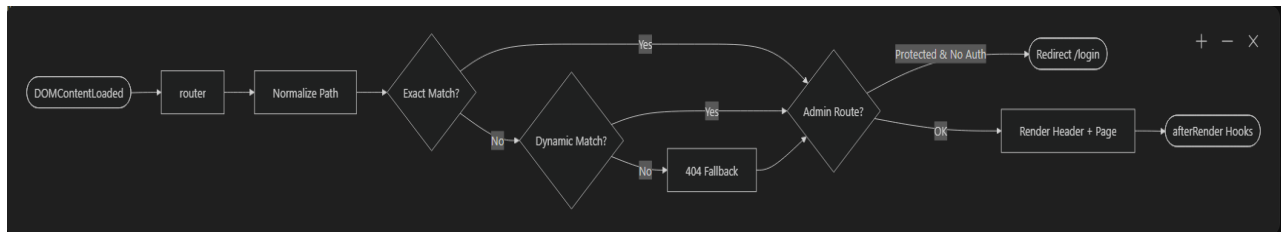
1.2 Single Page Application (SPA) and Client-Side Routing

Traditional multi-page navigation, which causes browser refreshes, was replaced with a custom client-side router.

- **Routing Logic:** The system is programmed to listen for URL changes (utilizing Hash-based routing or the History API). When a link is clicked, the router intercepts

the event, prevents default browser behavior, and dynamically injects the relevant Page Component into the main content area.

- **Seamless Transitions:** This architecture allows for a persistent Header and Sidebar, ensuring only the central content area updates. This creates a smooth experience where background processes are not interrupted during navigation.



1.2: Diagram of the SPA Routing Flow

1.3 Component-Based Structural Design

To ensure the codebase remains maintainable and scalable, a strict component-based directory structure was adopted:

- **Atomic Components:** Small, reusable UI elements such as Header.js, AdminSidebar.js, and specialized buttons. These are located in the src/components/ directory.
- **Page-Level Components:** Located in src/pages/, these files act as the "orchestrators" for specific views (e.g., MoviesPage.js, BookingPage.js). They manage data fetching, state for that specific view, and sub-component rendering.
- **Layout Management:** A flexible layout system was designed to automatically adjust based on the user's role. If an "Admin" is logged in, the AdminSidebar is injected; otherwise, the standard "Customer" navigation is displayed.

1.4 State Handling and Reactive UI Updates

Managing data within a Vanilla JS environment requires a disciplined approach to state:

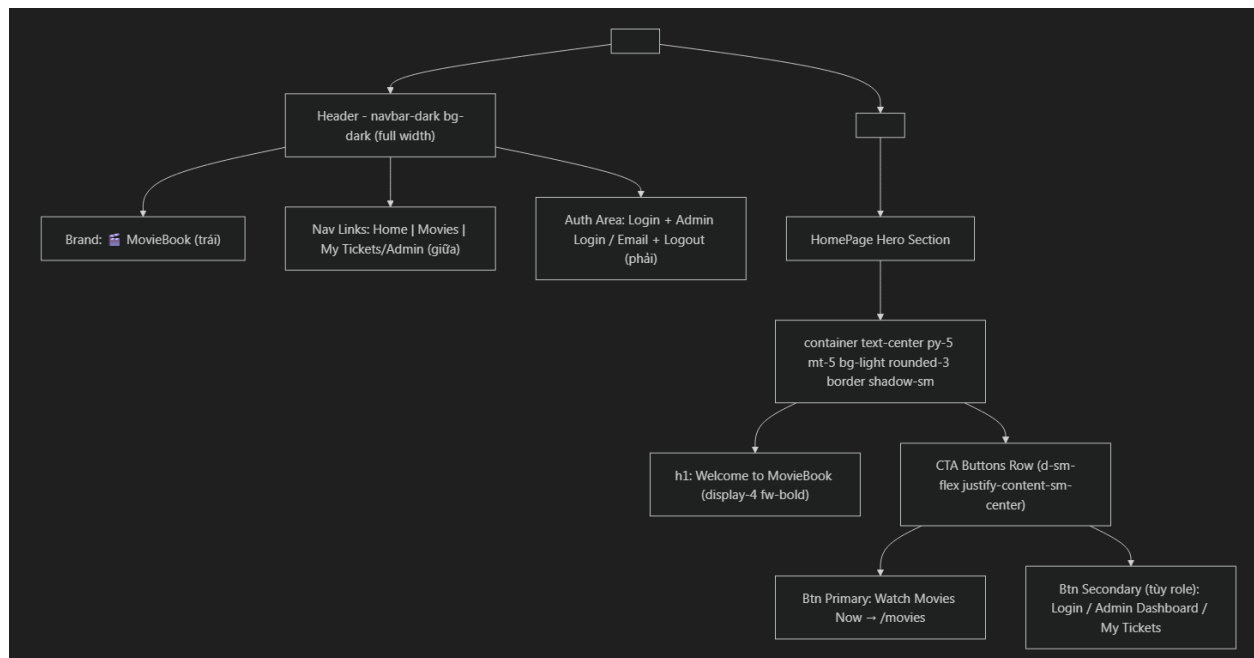
- **Local State:** Each page maintains its own state (e.g., currently selected seats or a filtered movie list). When the state changes, the UI is updated surgically using targeted DOM manipulation.
- **Global Persistence:** Critical information such as authentication tokens, email, and role are managed through a centralized state stored in localStorage, ensuring the session remains active across refreshes.

- **Event-Driven Updates:** JavaScript event listeners are used to handle user interactions (clicks, inputs, form submissions), ensuring that the UI responds instantly to user input.

1.5 Responsive Design and Accessibility

A core requirement was full functionality across devices ranging from large desktop monitors to smartphones.

- **Bootstrap Grid System:** The fluid container-row-col system was used to ensure movie posters and admin tables re-stack elegantly on smaller screens.
- **Mobile-First Approach:** Navigation menus were designed to collapse into "hamburger" menus on mobile, and the Seat Selection map was optimized for touch-screen interaction with adequate spacing between icons.
- **Visual Feedback:** Every interactive element provides clear feedback. Buttons change color on hover, loading spinners appear during API calls, and form validation messages appear in real-time if invalid data is entered.



1.5: Comparison of the Homepage

2. CLIENT INTEGRATION - THE API CLIENT LAYER

2.1 Centralized Communication Strategy

In a microservices-based project, the Frontend must interact with multiple backend services through a single entry point: the API Gateway. To manage this complexity, a specialized integration layer, `apiClient.js`, was developed to act as a "Middleman" between UI components and the backend server.

- **The DRY Principle:** By centralizing all `fetch()` calls into one file, the "Don't Repeat Yourself" principle was upheld, allowing for a single point of update for API URLs or security headers.
- **Request Abstraction:** Low-level complexities of the Fetch API (such as setting headers, converting bodies to JSON, and checking response status) were abstracted into high-level, easy-to-use functions.

```
// Base URL for API Gateway. It runs on port 5005 by default.
export const API_GATEWAY_URL = import.meta.env.VITE_API_BASE_URL || 'http://127.0.0.1:5005/api';

// Helper function to make fetch requests with Authorization header if available
export async function request(endpoint, options = {}) {
  const user = localStorage.getItem('user');
  const userToken = user ? JSON.parse(user).token : null;
  const userEmail = user ? JSON.parse(user).email : null;

  const headers = {
    'Content-Type': 'application/json',
    'peko-key': 'BO_CHIKA',
    ...(options.headers || {}),
  };

  // Add Authorization header if token exists
  if (userToken) {
    headers['Authorization'] = `Bearer ${userToken}`;
  }

  // Add X-User-Email header if email exists
  if (userEmail) {
    headers['X-User-Email'] = userEmail;
  }
}
```

2.3: The base configuration and default headers

2.2 Secure Header Management and Gateway Authentication

The project utilizes a multi-layered security approach at the Gateway level, which the apiClient.js satisfies for every outgoing request through automatic header injection:

- **Static API Key (peko-key):** Every request is automatically tagged with the unique API key to prevent unauthorized external access.
- **JWT Bearer Token:** For protected routes (like Booking or Admin actions), the client automatically retrieves the token from localStorage and attaches it to the Authorization header in the Bearer <token> format.
- **Identity Headers:** To assist microservices in identifying the current user, the client is capable of injecting X-User-Email and X-User-Role headers when available.

2.3 Standardized CRUD Method Wrappers

A suite of asynchronous methods within the apiClient was implemented to handle standard RESTful operations:

- **GET:** Optimized for fetching movie lists, showtime details, and profiles, including query parameter handling for searching and filtering.
- **POST:** Used for operations like User Registration, Login, and creating Bookings, ensuring Content-Type: application/json is correctly set.
- **PUT/PATCH:** Primarily used for updating movie details or showtime prices in the Admin Dashboard.
- **DELETE:** Designed for removing records, such as canceling a booking or an admin deleting a movie entry.

2.4 Robust Error Handling and Session Recovery

One of the most critical roles of the apiClient.js is managing the "Unhappy Path":

- **Global Status Checking:** Every response is checked for success. If a service is down (503) or a server error occurs (500), the client returns a structured error object for the UI to display.
- **Unauthorized Access (401/403):** If the Gateway returns a 401 Unauthorized status (meaning the token has expired), the apiClient is programmed to automatically clear the invalid session and redirect the user to the LoginPage.
- **Payload Validation Errors:** It captures 400-level errors from the backend (like "Seat already taken") and passes the specific error message back to the UI component.

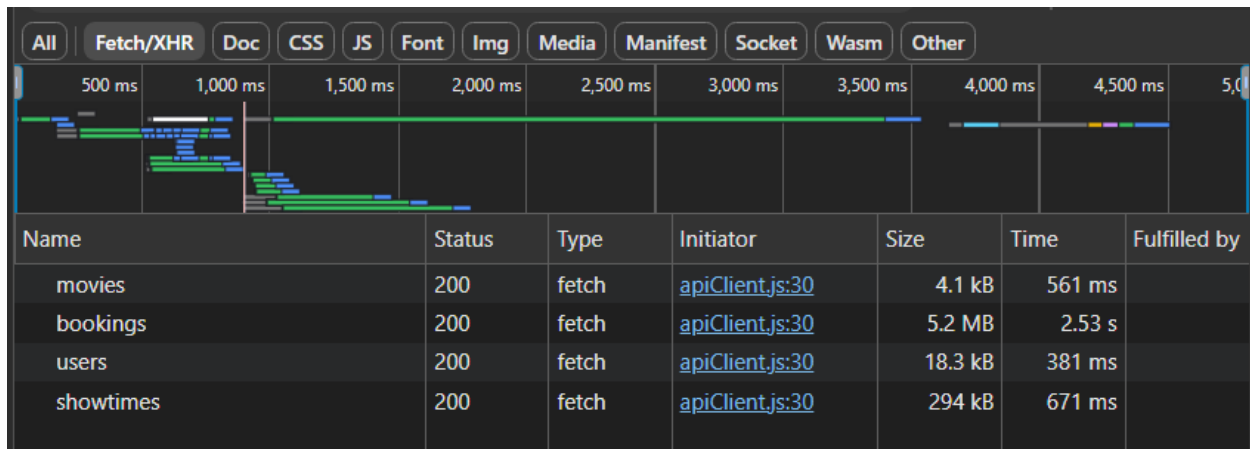


2.4: Data Flow Summary

2.5 Data Transformation and JSON Parsing

Since the backend microservices communicate exclusively via JSON, the apiClient handles the heavy lifting of:

- Checking if the response is actually JSON before attempting to parse it (preventing "Unexpected token" crashes).
- Standardizing the output format so that every Page component receives a predictable object containing { data, error, status }.
- Handling specialized requests, such as movie poster image uploads, which require FormData instead of standard JSON.



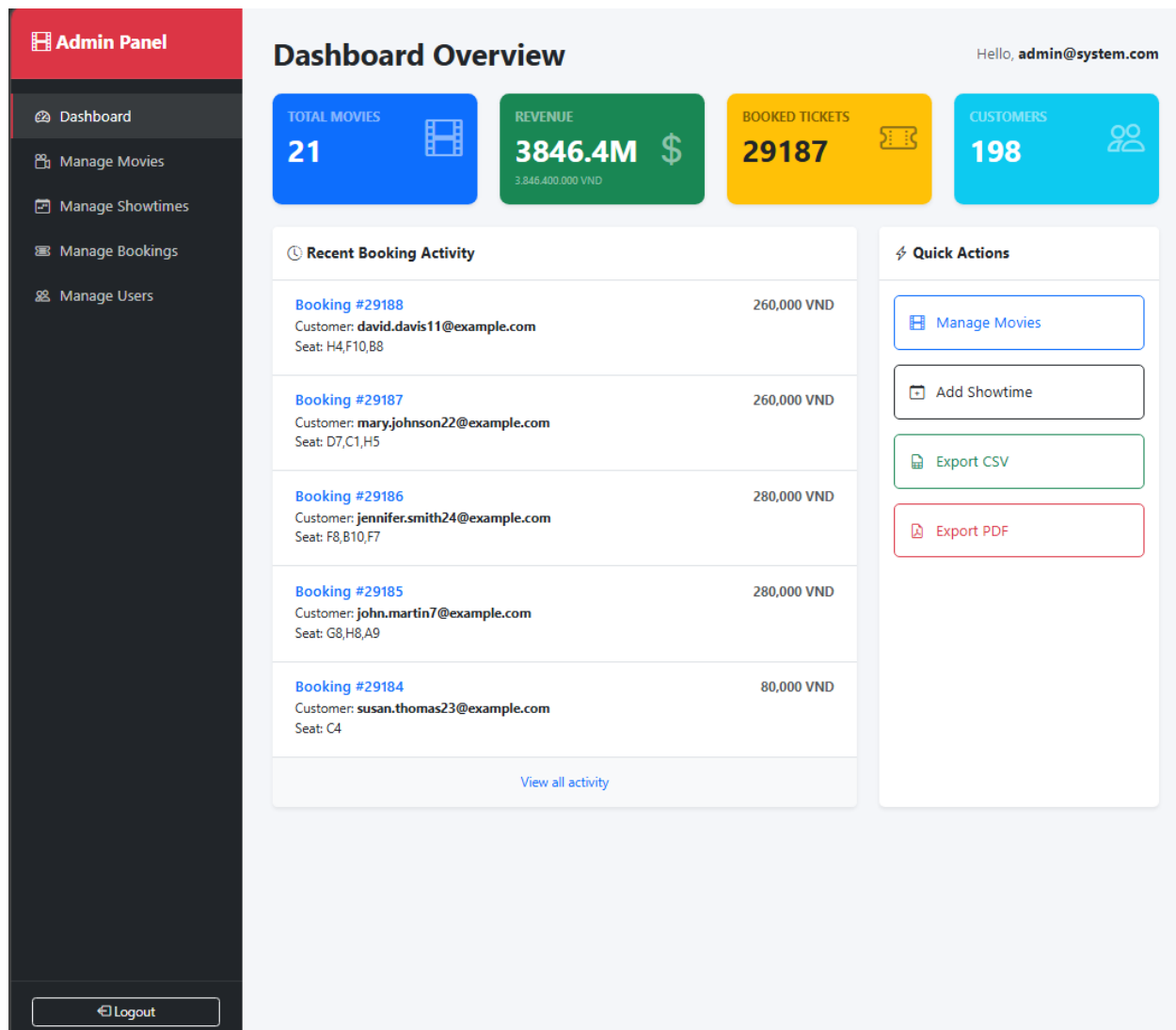
2.5: Successful API response being logged in the browser's Network Tab

3. ADMIN PAGES - MANAGEMENT MODULES & CRUD OPERATIONS

3.1 The Admin Dashboard Ecosystem

The Admin Portal was designed as a robust "Command Center" for theater managers, prioritizing data density, efficiency, and precise control over core resources.

- **Role-Based Access Control (RBAC):** Before any Admin page is rendered, the system verifies the user's role via the `apiClient.js`. Unauthorized users are immediately redirected.
- **Persistent Sidebar Navigation:** The `AdminSidebar.js` component was developed to provide quick access between different management modules without losing application state.



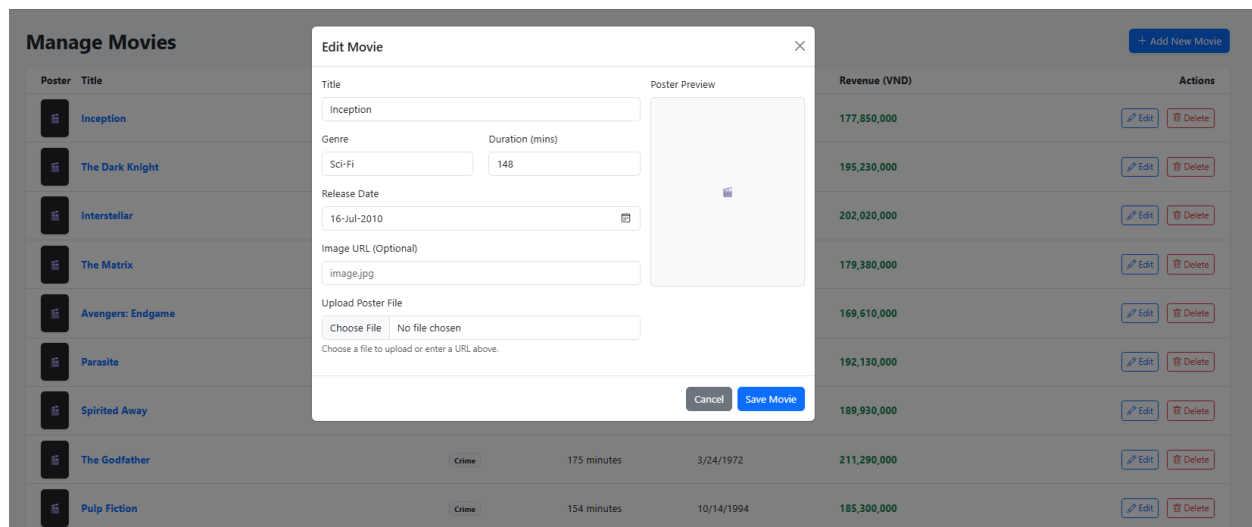
3.1: Full view of the Admin Dashboard

3.2 Movie Management and Resource CRUD

The **Movies module** serves as the primary administrative hub, managing the complete lifecycle of every film within the system through a comprehensive **CRUD** (Create, Read, Update, and Delete) workflow.

- **Dynamic Movie Table:** Utilizing **Vanilla JS**, a custom rendering engine was developed to process movie arrays from the backend into a sortable and searchable HTML table. This component integrates thumbnail previews of movie posters to facilitate rapid identification.

- **Movie Editor Modal:** To maintain a streamlined UI, a unified dynamic modal was implemented for both movie creation and modification. This modal supports complex data fields, including Title, Genre, Duration (in minutes), Release Date, and detailed Descriptions.
- **Image Upload Integration:** A specialized file-upload handler was engineered to enable administrative uploads of local **.jpg** or **.png** posters. This functionality necessitated custom logic within the apiClient to transmit **multipart/form-data** to the Movie Microservice.



3.2: Screenshot of the Movie Management table with Edit Movie

3.3 Showtime and Room Management

Showtime management serves as the critical link between the **Movies** and **Bookings** modules. Scheduling logic was implemented to ensure the process is both intuitive and error-resistant:

- **Showtime Scheduling:** Movies are assigned to specific rooms (e.g., Hall 1, Hall 2) with precisely defined Start and End times within the administrative interface.
- **Automatic Duration Check:** Client-side logic was integrated to alert administrators if a selected end-time is shorter than the actual duration of the movie, effectively preventing impossible scheduling scenarios.
- **Dynamic Price Controls:** The module enables global adjustment of ticket prices for specific screenings, with changes reflected immediately across the customer-facing Booking Flow.

- **Cascading Logic:** The system is designed so that the deletion of a showtime automatically triggers a "Refund & Notify" event via the backend (RabbitMQ). The frontend captures these events and provides real-time feedback to the admin through success alerts.

3.4 Dynamic Table Rendering and Search Optimization

To efficiently manage extensive datasets involving hundreds of movies or thousands of bookings, a custom **Dynamic Table** system was engineered to optimize both performance and user experience.

- **On-the-fly Filtering:** A search interface was developed to filter table rows in real-time based on administrative input. For small-to-medium datasets, this logic operates entirely on the client side, eliminating the need for redundant API calls and ensuring instantaneous results.
- **Template Literals for Rendering:** By leveraging **ES6 template literals**, a "Table-as-a-Component" pattern was established. This approach ensures that the UI updates only the specific rows affected by data changes, maintaining high browser responsiveness even during intensive data operations.
- **Status Indicators:** The integration of **Bootstrap "Badges"** provides immediate visual clarity for booking statuses (e.g., Confirmed, Pending, Cancelled) within the administrative tables. This implementation facilitates efficient and instant visual auditing of system records.

Manage Bookings

susan.brown32@example.com

ID	User Email	Showtime	Seats	Amount	Status	Actions
#29183	susan.brown32@example.com	ID: 3	D7,H2	160,000 VND	confirmed	Cancel
#29155	susan.brown32@example.com	ID: 13	A10,D3,B9,F2	200,000 VND	confirmed	Cancel
#28898	susan.brown32@example.com	ID: 92	C16,A3	200,000 VND	confirmed	Cancel
#28803	susan.brown32@example.com	ID: 122	A3	50,000 VND	confirmed	Cancel
#28563	susan.brown32@example.com	ID: 183	B4	100,000 VND	confirmed	Cancel
#28543	susan.brown32@example.com	ID: 190	G1,A1	200,000 VND	confirmed	Cancel
#28497	susan.brown32@example.com	ID: 208	C4	50,000 VND	confirmed	Cancel
#27928	susan.brown32@example.com	ID: 399	A5,H10	200,000 VND	confirmed	Cancel
#27253	susan.brown32@example.com	ID: 612	F11	50,000 VND	confirmed	Cancel
#26975	susan.brown32@example.com	ID: 695	E10	100,000 VND	confirmed	Cancel
#26959	susan.brown32@example.com	ID: 697	C8,C11,G4,E3	420,000 VND	confirmed	Cancel
#26895	susan.brown32@example.com	ID: 722	A9	50,000 VND	confirmed	Cancel
#26623	susan.brown32@example.com	ID: 807	F4,H9	220,000 VND	confirmed	Cancel
#26539	susan.brown32@example.com	ID: 835	G8,G11,D7	230,000 VND	confirmed	Cancel
#26059	susan.brown32@example.com	ID: 987	A8	50,000 VND	confirmed	Cancel
#25854	susan.brown32@example.com	ID: 1044	B11,G2	140,000 VND	confirmed	Cancel
#25771	susan.brown32@example.com	ID: 1074	C7,G3,C16	220,000 VND	confirmed	Cancel

3.4: Search/Filter functionality in the Admin Bookings

3.5 Form Validation and Data Integrity

To prevent invalid or "garbage data" from entering the database, a rigorous validation layer was implemented within the administrative forms.

- **Real-time Validation:** Input fields utilize visual feedback, such as turning red and displaying specific error messages, when mandatory fields like movie titles are left empty or when invalid date formats are detected.
- **Logic Constraints:** Within the "Showtimes" module, a specialized date-picker integration was developed to prevent the selection of dates in the past, ensuring scheduling accuracy.
- **Confirmation Dialogs:** To mitigate the risk of accidental data loss during high-stakes administrative tasks, custom confirmation prompts are triggered before any "Delete" action is executed.

3.6 Analytics and System Overview

The primary Dashboard (AdminDashboardPage.js) incorporates an "At-a-glance" analytics section for centralized oversight. Key Performance Indicators (KPIs) are displayed using modular card components to provide immediate insights into the system's status:

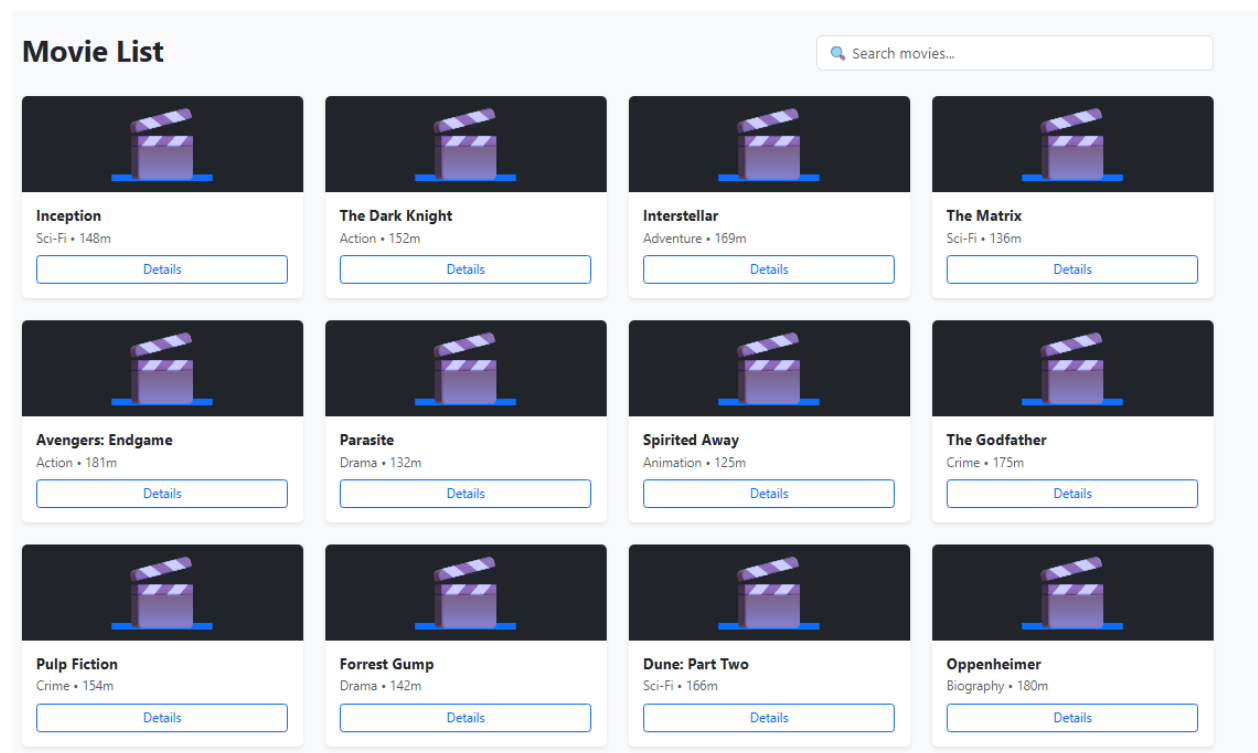
1. **Total Movies:** The current count of films actively showing in the system.
2. **Active Showtimes:** A summary of all scheduled screenings for the current day.
3. **Recent Bookings:** A real-time count of late-stage transactions and reservations.
4. **System Health:** A dedicated status indicator that monitors and displays the connectivity status between the API Gateway and underlying Microservices.

4. USER PAGES - CUSTOMER JOURNEY AND INTERACTIVE BOOKING FLOW

4.1 The "Cinema First" Visual Experience

The public-facing side of the application features a "Media-Rich" design to prioritize a high-conversion environment.

- **Dynamic Hero Section:** A featured movie section highlights the latest blockbuster with high-resolution backdrops and clear Call-to-Action (CTA) buttons.
- **Search and Discovery:** A real-time search bar filters the movie list by title or genre, providing an "Instant Search" feel through DOM updates.
- **Loading States:** "Skeleton Screen" placeholders mimic the layout of movie cards before data is fully loaded.



4.1: Movie Grid in user account

4.2 Movie Details and Information Hierarchy

The MovieDetailsPage.js presents a clean hierarchy of metadata, including Title, Genre Badges, Duration, and a Synopsis.

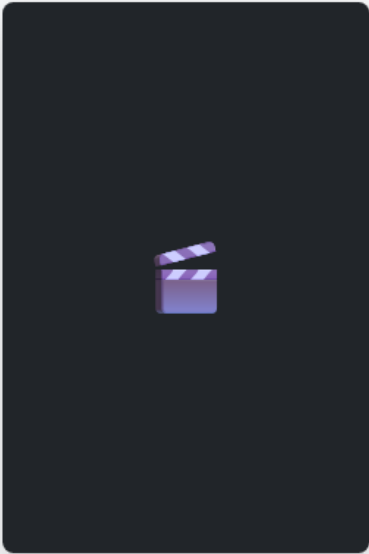
- **Showtime Listing:** All active showtimes for a specific movie are dynamically fetched and presented in a "Card-based" selection system.
- **Conditional Rendering:** If no showtimes are available, the booking section is hidden and a "Coming Soon" message is displayed.

MovieBook

HomeMoviesMy Tickets

Hello, user@example.comLogout

Movies / Details



Inception

Sci-Fi148 minutesRelease Date: 7/16/2010

No description available for this movie.

Showtimes Schedule

< Previous

February 2026

Next >

Sun	Mon	Tue	Wed	Thu	Fri	Sat
1 3 shows	2 2 shows	3 2 shows	4 3 shows	5 2 shows	6 2 shows	7 3 shows
8 3 shows	9 3 shows	10 3 shows	11 2 shows	12 2 shows	13 3 shows	14 2 shows
15	16	17	18	19	20	21
22	23	24	25	26	27	28
1	2	3	4	5	6	7
8	9	10	11	12	13	14

Showtimes for Thursday, 8 January 2026

09:00
Standard Hall 2
50,000 VND

14:00
Standard Hall 1
50,000 VND

20:45
Standard Hall 3
70,000 VND

4.2: Movie Details Page and available showtimes

4.3 The Interactive Booking Flow (The Core Engine)

The `BookingPage.js` manages the complex state of hall layouts and seat availability.

- **The Virtual Seat Map:** A dynamic seat-grid generator renders the layout of the cinema hall based on Room Microservice data.
 - **Visual Logic:** CSS classes are used to distinguish between Standard, VIP, and Couple seats.
 - **State Tracking:** A "Selection Buffer" is maintained in the local state, and booked seats are disabled.
- **Real-time Pricing Engine:** A live calculator sums the base showtime price with seat-specific surcharges, updating the "Total Price" instantly.
- **Atomic Reservation Logic:** An "Availability Check" is performed via the API before proceeding to payment to ensure seats remain vacant.

 **MovieBook**

Home Movies My Tickets

Hello, user@example.com [Logout](#)

Seat Map

Screen at the front - Please select seats

SCREEN

A	1	2	3	4	5	6	7	8	9	10	11	A
B	1	2	3	4	5	6	7	8	9	10	11	B
C	1	2	3	4	5	6	7	8	9	10	11	C
D	1	2	3	4	5	6	7	8	9	10	11	D
E	1	2	3	4	5	6	7	8	9	10	11	E
F	1	2	3	4	5	6	7	8	9	10	11	F
G	1	2	3	4	5	6	7	8	9	10	11	G
H	1	2	3	4	5	6	7	8	9	10	11	H

1-2 3-4 5-6 7-8 9-10

☐ Available

☒ Selected

☐ Booked

☐ VIP (+20k)

☐ Couple (+50k)

BOOK TICKET

IMAX Hall 1

Selected seats:
E5, E6, E7

Subtotal:
360,000 VND

Book Now

OTHER SHOWTIMES

Select Date

Sun 1/2

Mon 2/2

Tue 3/2

Wed 4/2

Thu 5/2

Fri 6/2

Sat 7/2

Sun 8/2

10:45

IMAX Hall 3 | 100,000 VND

11:30

IMAX Hall 1 | 100,000 VND

[← Back to movie list](#)

4.3: The Seat Selection Map

4.4 Payment Integration and Security

The PaymentPage.js handles the final step of the transaction with specialized UX features:

- **Secure Form Masking:** A mask-logic was written to only display the last 4 digits of credit card numbers, mirroring professional banking standards.
- **Payment Method Management:** A "Wallet" feature allows users to save, view, or delete payment methods via the Payment Microservice.
- **Transaction Feedback:** Successful payments lead to a high-fidelity "Success" screen with the Booking ID.

4.5 User Profile and "My Bookings"

The ProfilePage.js and MyBookingsPage.js modules provide account management and purchase history.

- **Booking History:** This module renders "Digital Tickets" for historical and upcoming bookings, including Movie Title, Hall, and Seat Numbers.
- **Account Management:** Users can update profile information and passwords with integrated form validation.
- **Actionable History:** "Cancel" buttons for upcoming bookings trigger refund processes through the backend and update the UI state instantly.
-

MovieBook

[Home](#) [Movies](#) [My Tickets](#)

Hello, user@example.com

Logout

My Booked Tickets

Booking ID	Showtime	Seats	Price	Status	Actions
#145	ID: 1267	C1	50,000 VND	Paid	Cancel Ticket
#167	ID: 330	H6	90,000 VND	Paid	Cancel Ticket
#203	ID: 161	E7	70,000 VND	Paid	Cancel Ticket
#221	ID: 800	E1	100,000 VND	Paid	Cancel Ticket
#432	ID: 1146	I10	70,000 VND	Paid	Cancel Ticket

4.5: My Tickets page

5. SUMMARY OF CONTRIBUTIONS

5.1 Technical Excellence and Architectural Integrity

The development of the frontend was governed by a commitment to high-performance architecture and modularity. By leveraging a Single Page Application (SPA) framework built with Vanilla JavaScript (ES6+) and Vite, the project achieved near-instantaneous load times and a highly responsive user interface without the bloat associated with traditional heavy frameworks.

- **Performance Optimization:** The elimination of unnecessary library dependencies resulted in a significantly reduced bundle size, ensuring that the application remains lightweight and fast even on lower-end hardware or limited mobile networks.
- **Scalability through Modularity:** The adoption of a component-based structure—separating atomic UI elements from complex page orchestrators—provided a codebase that is easily extensible. New features, such as additional payment methods or new admin modules, can be integrated with minimal disruption to existing logic.

5.2 Robust Security and Seamless API Integration

A cornerstone of the contribution was the creation of a centralized, secure communication layer via `apiClient.js`. This module successfully bridged the gap between the frontend and the Microservices API Gateway, ensuring that data integrity and security were never compromised.

- **Security Standardization:** Through the automatic injection of JWT Bearer tokens and custom API keys (`peko-key`), a uniform security posture was maintained across all outgoing requests. This centralized logic effectively shielded the application from common vulnerabilities and ensured that only authorized users could access sensitive administrative or booking functions.
- **Error Resilience:** The implementation of advanced error-handling patterns—including automatic session recovery for expired tokens and structured feedback for server-side failures—significantly improved system reliability. Users are provided with clear, actionable information rather than generic error messages, enhancing trust and usability.

5.3 Administrative Empowerment and Data Management

The delivery of a comprehensive Admin Portal transformed the backend data into an actionable management tool. By prioritizing a "Data-First" design, the administrative experience was optimized for high-volume operations.

- **Efficient CRUD Workflows:** The design of dynamic tables and multi-purpose modals allowed for the rapid management of movies, showtimes, and users. These tools reduced the time required for routine tasks such as updating ticket prices or scheduling new film releases.
- **Data Integrity and Validation:** Rigorous client-side validation ensured that only clean, well-formatted data reached the microservices. This proactive approach to data quality reduced the load on backend services and prevented database inconsistencies caused by human error.

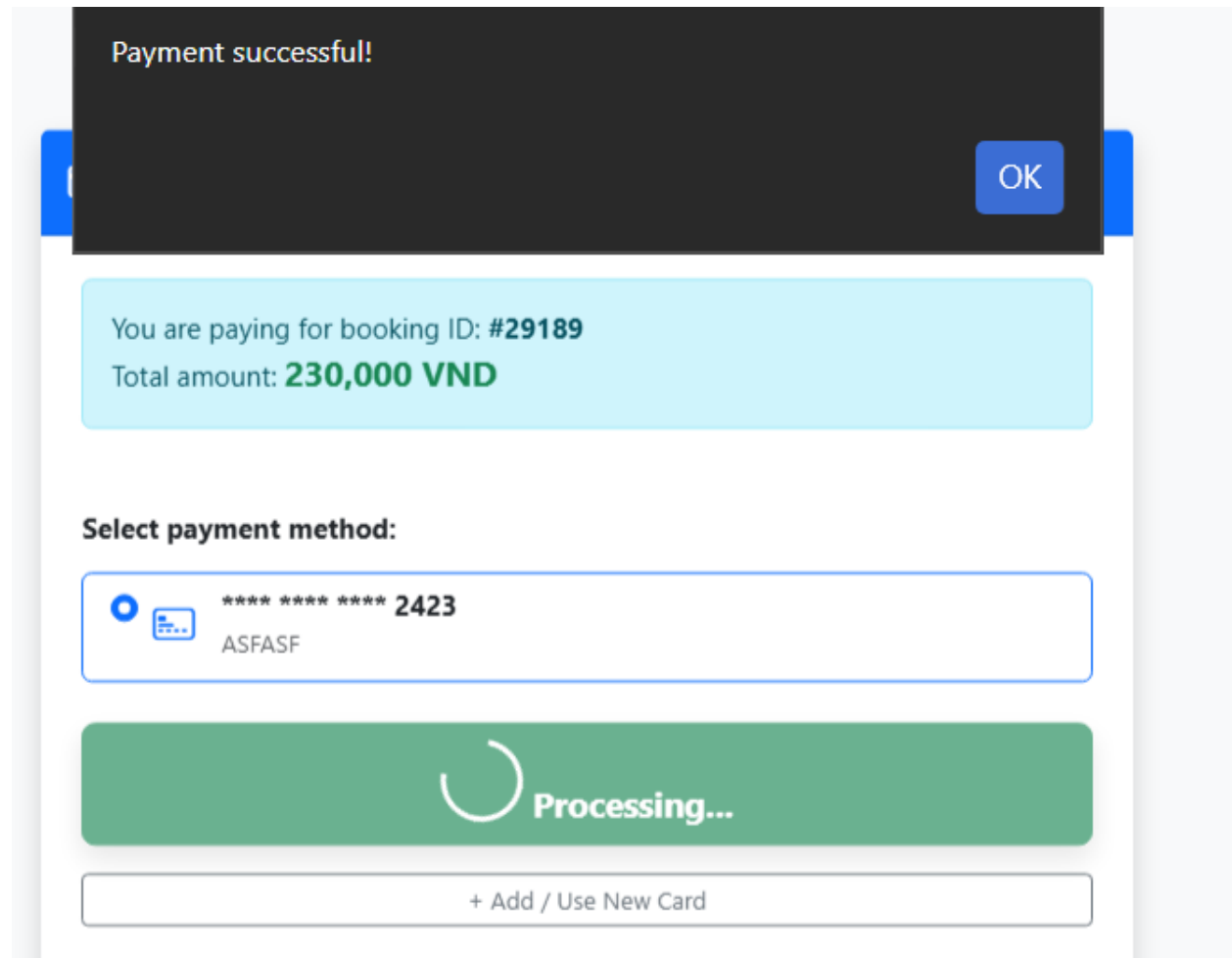
5.4 User-Centric Design and Conversion Optimization

The public-facing user experience was engineered to drive engagement and facilitate a frictionless booking journey. Every visual element was designed with the end-user's needs at the forefront.

- **Interactive Booking Engine:** The development of the virtual seat map and real-time pricing engine provided a sophisticated "Live" feel to the booking process. The ability to see surcharges for VIP or Couple seats updated instantly encouraged transparency and informed decision-making for the customer.
- **Inclusive Responsive Design:** By employing a Mobile-First strategy and a fluid grid system, the application reached a broader audience. The consistent experience across desktop, tablet, and mobile platforms ensured that users could book tickets anytime, anywhere, with total ease of use.
- **Professional UX Polish:** The inclusion of micro-interactions—such as toast notifications, loading skeletons, and secure card masking—elevated the application to a professional industry standard, matching the quality of major commercial cinema platforms.

5.5 Conclusion of Impact

The contributions described in this report represent a holistic approach to modern frontend engineering. By balancing technical performance, rigorous security, administrative utility, and a premium user experience, a foundation was built that successfully supports the complex requirements of a distributed microservices architecture. The final product stands as a visually appealing, highly functional, and secure gateway to the Movie Ticket Booking System, fulfilling all project objectives with technical precision.



5.5: Payment successful pop-up